IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received January 28, 2020, accepted February 25, 2020, date of publication April 3, 2020,
date of current version December 29, 2020.

*Digital Object Identifier 10.1109/ACCESS.2020.2985501*

# An Efficient and Fast VLIW Compression Scheme for Stream Processor

**GONGLI LI, YINGYING HOU, AND JUNZHE ZHU**
College of Computer and Information Engineering, Henan Normal University, Xinxiang 453007, China
Big Data Engineering Lab of Teaching Resources and Assessment of Education Quality, Xinxiang 453007, China

Corresponding author: Gongli Li (ligl522@163.com)

This work was supported in part by the Science and Technology Research Project of Henan Province under Grant 192102210131, and in part by the Doctoral Research Project of Henan Normal University under Grant 5101119170144.

**ABSTRACT** Stream processor has been widely used in multimedia processing because of the high performance gained by parallelism. In order to achieve higher parallelism, the stream processor employs large width structure of VLIW (very long instruction word, VLIW) and multiple parallelizable instructions are organized into one VLIW. Because the width of VLIW is fixed, there are a large number of empty operations (non-operation, NOP) filled in VLIW, which results in serious code size expansion problem. Aiming at this issue, the horizontal code compression and vertical code compression methods are applied on the VLIW of stream processor respectively. First the VLIW is divided into several subfields according to the logic characteristics of VLIW instruction, then the horizontal code compression scheme which based on Huffman coding is applied on each subfield and this method can achieve approximately 78% code size reduction on average. However, the extra-long time required to decode the compressed VLIW before instruction execution may cause system performance penalty. In order to reduce the decompression time consumption, the vertical compression scheme is proposed. The vertical compression can reduce the code size nearly 70% by deleting the NOPs of VLIW in vertical direction. Furthermore the VLIW after vertical compression can be executed directly without decompression operation by using banked instruction memory. Specifically, the vertical compression can compress stream processor VLIW code size significantly and without any negative influence on performance.

**INDEX TERMS** Stream processor, Huffman coding, horizontal compression, vertical compression, compression ratio.

## I. INTRODUCTION

The stream processors [1]–[5] have achieved high performance through integrating a lot of parallel function units to develop the parallelism of application and have been successfully used in multimedia processing [6]–[8], mobile devices [9]–[11] and information security [12], [13]. The stream processors adopt VLIW to exploit the parallelism of different function units. The number of operations in one VLIW is fixed and each operation corresponds to one function unit. But due to the reasons, such as the limitation of inherent ILP (instruction level parallelism, ILP) in the stream program, the capability of the compiler, data correlation, resources constraints and so on, the VLIW is not always replete with valid operations. However, the VLIW is of fixed width, lots of NOPs (non-operation) have to be filled in VLIW, which increases the code size sharply. In this situation, more instruction memory would be occupied and more access bandwidth would be wasted. That's why stream processors have to use high-capacity instruction memory. For example, the instruction memory of Imagine [1] and MASA [2] are 144KB and 160KB respectively, occupying 11% and 27% of the whole on-chip area.

In order to reduce the VLIW code size and improve the VLIW instruction density, data compression technology is used in code compression. Data compression technology can fall into lossy compression [14], [15] and lossless compression [16]. For image and video, lossy compression is used by removing the information that human can't perceive. But for code compression, lossless compression must be used because any instruction information loss after decompression will lead to wrong results. The lossless

The associate editor coordinating the review of this manuscript and approving it for publication was Junaid Shuja.

compression—entropy coding [17] generates variable-length code based on information entropy, represented by Huffman coding [18], [19] and arithmetic coding [20].

Huffman coding, as one of the optimal coding schemes, could be close to the maximum theoretical CR (compression ratio). Wolfe *et al.* [21], first put forward the idea of code compression, and then used Huffman coding to compress MIPS instruction set. The CR of Testbench could reach 73%. Yang [22] proposed a novel correlations model which combined type model and position model for embedded processor RISC instruction. Through applying this approach, the best CR could reach 53.16%. Dias *et al.* [23] proposed a new code compression method by using Huffman-based multi-level dictionary for embedded systems and reduced code size up to 30.6%. As the Huffman coding applied to MIPS and RISC instruction can reduce the code size effectively. Bonny *et al.* [24]applied the Huffman coding to compress code for embedded system and achieved the average compression ratio of 48%. Rahman *et al.* [25], [26] proposed two lossless compression schemes for image, the first one is based on Huffman coding and obtained a compression ratio of 44.8%, the other one is based on Huffman coding and binary coding, and achieved execution time reduction.

Arithmetic coding has two basic parameters: probabilities of symbols and their coding intervals. Wei *et al.* [27] applied the arithmetic coding to transport triggered architecture and achieved the compression ratio of 35.7%, but it didn't mention any detail about decoding cost. Paridaens *et al.* [28] proposed several genomic data compression methods based on arithmetic coding and in the best case, the compression ratio is 21%, but the decompression process is time-consuming.

Buzdar *et al.* [29] focused on the decompression overhead of application-specific, compressed instruction format, and designed the instruction decompressor for VLIW processor, which improved the efficiency in terms of power, area and delay. Wang *et al.* [30] used variable length code in line buffer architecture to improve the compression performance and achieve the 50% compression ratio. Lamire *et al.* [31] applied the byte-oriented compression technique to arrays of integer data and used stream Vbyte decoding method to enhance the performance of decompression. Therefore, when designing a compression scheme, we must consider not only the compression ratio, but also the overhead of decompression.

The previous compression schemes can achieve excellent compression ratio, but the decompression speed and its penalty are not discussed. As for stream processor, the processing performance is the most important goal. In order to increase the compression ratio, at the same time enhance the decoding speed and reduce the impact on system performance after code size compressed, an efficient and fast VLIW compression scheme for stream processor is proposed.

The contributions of our work can be concluded as follows: a) After in-depth analysis of VLIW characteristics on stream processor, it is proposed to divide a VLIW instruction into multiple subfields, which can effectively increase the probability of NOPs and improve the compression ratio; b) The horizontal compression and vertical method are both applied to VLIW. In vertical compression, it only deletes NOPs in the vertical direction without changing the structure of the instruction. Therefore, it does not need to be decompressed before execution, and it will not affect the performance of the stream processor; c) The banked instruction memory structure and address calculation unit are designed to realize the rapid reorganization of instructions.

The rest of this paper is structured as: in section 2 the characteristics of VLIW running on stream processor were analyzed. Then according to the logic feature of VLIW, each VLIW was divided into several subfields and adopted horizontal compression to compress every subfield in horizontal direction in section 3. As the decoding speed after horizontal compression turned to be very slow, in section 4, the vertical compression scheme was proposed. Section 5 was experiment and result analysis of vertical compression. The last section concluded this paper.

## II. VLIW CHARACTERISTICS ANALYSIS

The program running on stream processor consists of a series of computing core called kernel, and every kernel consists of multiple VLIWs. The format of the kernel can be represented in Figure 1, in which a row can be seen as a VLIW, a rectangle is an operation, the gray rectangle indicates it's a valid operation while the white rectangle indicates it's a NOP.

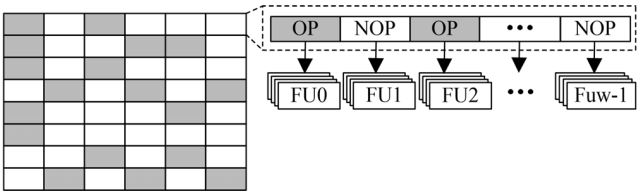



**FIGURE 1.** A kernel composition and the VLIW format.

Based on the research and development of stream processors CISP (computing-intensive stream processor, CISP) and after deep analysis of VLIW running on it, this paper found that the VLIW on stream processor architecture has the following features:

(1) The width of VLIW is large. For example, the instruction width of Imagine is 576-bit, MASA 640-bit and CISP 512-bit. That's because stream processors all integrate a large number of parallel functional units to achieve high performance and each unit corresponds to several bits of VLIW, thus leading to a large VLIW width.

(2) The number of NOPs in VLIW is huge. As every free functional unit in VLIW corresponds to a NOP, there are many idle function units in different applications. NOPs statistics in such typical steam applications as Reed-Solomon and H.264 suggests that the NOP percentage in every field is over 60% (as shown in Figure 2).

Based on the above characteristics, the horizontal compression method which based on Huffman coding is applied to VLIW to reduce the code size, as Huffman coding can

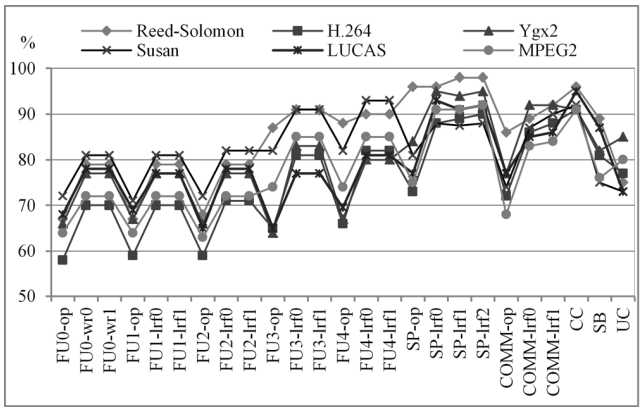



**FIGURE 2.** NOPs statistics in six typical steam applications.

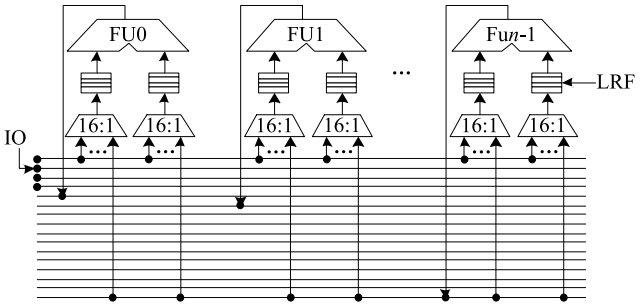



**FIGURE 3.** Structure of stream processor.

| Opcode | LRF0 rd | LRF1 rd | LRF0 bus | LRF0 wr | LRF1 bus | LRF1 wr |
|---|---|---|---|---|---|---|

**FIGURE 4.** Instruction format of a function unit.

compress the code size by replacing frequently used code with short code. Since the use probability varies from code to code, the greater the probability difference is, the better compression result can reach. Figure 2 illustrated that the use probability of NOP is higher than other operations, so compressing VLIW of stream processor with Huffman coding is feasible. As the Huffman coding method compresses the VLIW instruction in horizontal direction, it can be seen as horizontal compression.

## III. HORIZONTAL COMPRESSION

### A. VLIW DIVISION AND COMPRESSION

Because the VLIW width is large, the VLIW instruction has to be divided into several fields before compression, and each field needs to be encoded respectively. The dividing method of VLIW divided will directly affect the compression effect, and the reasons are as follows:

According to Huffman coding scheme, the average instruction length after compressed $L$ is:

$$L = \sum_{i=1}^{n} p_i \times l_i \tag{1}$$

$p_i$ is the use probability of instruction $i$, and $l_i$ is the length of instruction $i$ after Huffman coding. It is obvious that assigning shorter code to more frequently used instruction can reduce the average instruction length $L$. According to the statistics in Figure 2, the NOP is used most frequently in each field, so the code length of NOP can be assigned 1 and the use probability of the NOP is $p_1$, then

$$L = p_1 + \sum_{i=2}^{n} p_i \times l_i \tag{2}$$

Let the average coding length of other $n$-1 valid instructions be $m(m > 1)$, then the average length of VLIW after compression can be described as following:

$$L = m - (m - 1) \times p_1 \tag{3}$$

It can be seen from the above formula that promoting the NOP proportion of $p_1$ can lead to a shorter VLIW length $L$, thus favoring code size compression.

For an instruction can be determined as NOP, the instruction value must be all "0". The wider the instruction is, the less likely the instruction value is all "0", and the less likely it is NOP. For example, the instruction "00001010" is obviously not a NOP, but if it is divided into "0000" and "1010", then the first one is NOP. The instructions of every function unit in VLIW are more than 30-bit, so it can be divided into several subfields.

The architecture of steam processor, which is shown in Figure 3, has several function units and each function unit has its LRF (local register file, LRF) and an output bus. Operands are read from the LRF, and processing results are output to its own bus, then the other function units which need the results can read it from the output bus directly and write them to LRF.

According to this architecture of stream processor, the instruction of each function unit includes operation code, LRF read address, written data and LRF written address, as shown in Figure 4 (function unit with double operands as example).

If the function unit executes a valid operation, its instruction must have operation code and operands. In other words, operation code and LRF reading address are always valid or not valid at the same time, so we can take them as a subfield. The write operation must have write data and LRF write address, which means these two parts are associated, and they can be organized as a subfield. The read operation and write operation are independent and the two write operations are also independent, so according to the above logic characteristics of the stream processor instruction, every instruction can be divided into 3 subfields, one operation code subfield and two write operation subfields, as shown in Figure 4. Division instruction of each function unit can increase the probability of the NOP, which can benefit the compression operation obviously.

In Huffman coding, the instruction and its compressed code need to be stored in a code table. After one VLIW is divided into several subfields according to its logic characteristics, the width of each subfield is not equal. Assuming each subfield width as $l_1$, $l_2$,……, $l_k$ bit respectively, the capacity of code table is $\sum_{i=1}^{k} \boldsymbol{l_i} \times \mathbf{2}^{\boldsymbol{l_i}}$. The large table not only increases the size of the compressed code, but also increases the delay of lookup table in decompression operation.

As every field width is the multiple of 8 after dividing the instruction according to its logic characteristics, we divided each field into several 8-bit width subfields furthermore and applied fixed-length Huffman coding to each subfield. In this way, the compression process is simplified and the capacity of code table is only8 × 256bit.

After dividing one VLIW into several subfields, the probability of each instruction value is computed respectively and Huffman coding is applied to several typical stream applications. The CR is computed by formula (4) and the result is shown in Table 1:

$$CR = \frac{compressed\ code\ \text{size}}{pre - compressed\ code\ size} \times 100\% \qquad (4)$$

**TABLE 1.** CR of Huffman coding.

| Application Name | CR |
|---|---|
| Reed-Solomon | 19.49% |
| H.264 | 23.98% |
| Ygx2 | 21.87% |
| Susan | 20.61% |
| LUCAS | 22.36% |
| MPEG2 | 23.01% |
| **average** | 21.89% |

It can be seen from Table 1 that the average code size of six typical stream programs is reduced to 21.89% by Huffman coding. The main reason is that we improved the proportion of NOP by dividing instruction into multiple subfields according to its logical meaning. What's more, the fixed-length Huffman coding can effectively reduce the size of code table.

As the instruction formats for all function units are similar, we take function unit 0 as an example, and estimate the NOP proportion before and after division. The result is shown in Figure 5.

### B. DECOMPRESSION

The code size has been compressed sharply by Huffman coding. The compressed instructions have to be decompressed before execution. As the compressed instruction is of variable length after Huffman coding, the decoding operation must work in sequence. That is to say, for an input code stream, when a code is matched successfully in code table, this code scanning operation is ended and the start position of the next code could be determined. For example, an input code stream is "001011000", and the code table is as shown in Table 2:

In the decompression process, the input code stream is scanned from its beginning. When it scans to the third bit,

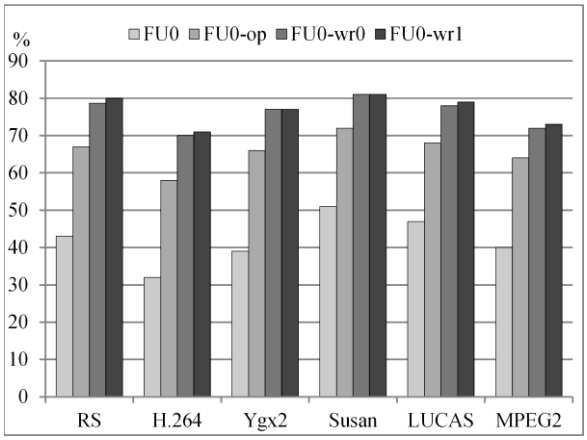



**FIGURE 5.** NOP proportion of pre- and post-dividing.

**TABLE 2.** Huffman code table.

| Value | Code |
|---|---|
| 00000000 | 1 |
| 11000110 | 01 |
| 11110000 | 001 |
| 10101010 | 000 |

the code is "001", and this code can match successfully in the code table. Only at this time, we can begin to scan the next code, when scan to "01", it matched the value "01" in the code table. The above decompression process implies that every code must be decompressed in sequence because of the variable-length code.

Acasandrei and Neag [32] and Zhou *et al.* [33] proposed adding head stream information respectively, taking parallel multi-way detection and pipelining technology to improve the decoding performance up to 1 code/cycle. Because the VLIW width of stream processor is 512-bit, and each subfield is 8-bit, a VLIW contains 64 subfields, so this means that a compressed VLIW contains 64 compressed codes. Even if the parallel decoding scheme is put forward in [32], [33], but a compressed VLIW still needs 64 cycles to decompress. This decompression speed couldn't match the instruction execution speed of stream processor and would cause serious stop-wait problem, which will greatly lower the system performance.

## IV. VERTICAL COMPRESSION

Huffman coding could reduce the code size effectively, but the compressed subfields have to be decompressed sequentially, that is time-consume and will cause great harm to system performance. In order to solve this problem, the vertical compression scheme is proposed.

### A. PROCESS OF VERTICAL COMPRESSION

It takes long time to decode one VLIW after horizontal compression mainly because (1) the width of VLIW instruction is large, so each VLIW has many subfields; (2) the nature of Huffman coding determines that all compressed codes

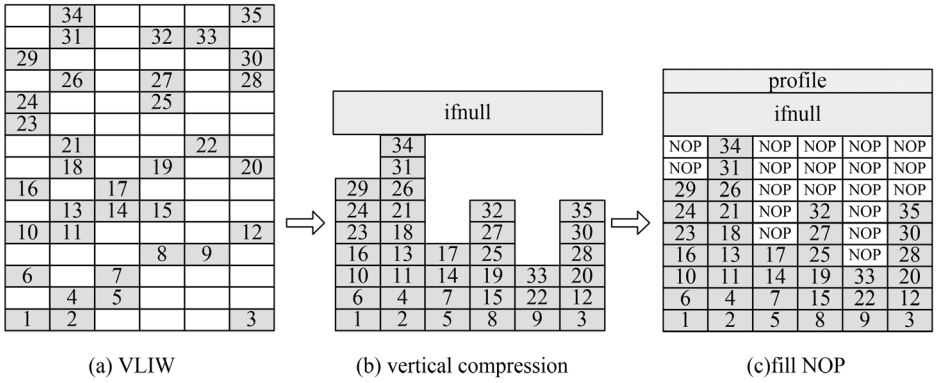



**FIGURE 6.** The process of vertical compression.

must be decoded in sequence. For the VLIW width is determined by the architecture of stream processor itself, which is unchangeable, so a parallel method for decompression has to be found to improve the decoding speed.

Huffman decoding must be in sequential, because both the length and the position of code are changed after Huffman coding and all compressed codes must be sequentially scanned and decoded. For the above problems, we designed a novel compression scheme: (1) compress VLIW in vertical direction; (2) in compression, NOP is deleted only and there isn't any change to the valid instruction. The procedure of the vertical compression is as follows.

First, a VLIW is divided into several subfields according to the method in section 3.1. For the whole stream application kernel, it is divided into several columns, as shown in Figure 6 (a).

Take each column as a subfield, and determine every subfield to be a NOP or a valid instruction from top to bottom. If the subfield is a NOP, delete it and the valid instructions move down. Then an array ifnull [*h*] [*w*] is used to record the corresponding subfield as NOP or valid instruction, in which *h* denotes the number of VLIW that a kernel has and w is the number of subfields that a VLIW contains. If the *j*th field of the *i*th instruction is NOP, then ifnull[*i*] [*j*]=0, or else the value is 1. The vertical compression result of a kernel is shown in Figure 6 (b).

Because the number of valid operations varies from columns, they are not necessarily equal after vertical compression. We have to fill NOPs into columns to make sure that the height of every column is equal before the compressed kernel is stored. Moreover, the compressed kernel has two kinds of information: compressed VLIW and ifnull array. In order to distinguish them, a profile containing the length of instruction and array is added at the beginning of kernel. After NOP filling, the result is shown in Figure 6 (c).

### B. VLIW LOADING AND EXECUTION

When a kernel needs to be executed, its VLIW instructions have to be loaded into instruction memory at first. The loading process is as follows.

(1) Read the profile of the kernel, and then determine the length of ifnull array and instruction according to the profile.

(2) The ifnull array is loaded into the FIR (field index register, FIR), and the data width of register is equal to the number of subfields that a VLIW has.

(3) The structure of vertically compressed VLIW is the same as the uncompressed structure, so it can be loaded to instruction memory directly. Because the width of VLIW is large, the instruction memory is banked (the width of each bank is the same as subfield width).

When the VLIW after vertical compression has been loaded into instruction memory completely, the kernel can begin to execute at once without decompression operations and the subfield address of every bank is computed by address generating unit, as shown in Figure 7.

FPCR (field PC register, FPCR): store the subfield offset addresses of current instruction in each bank, the initial value being 0.

BAR: (base address register): when a kernel is loaded into instruction memory, each field has a beginning address in its corresponding bank, and this information has been stored in the BAR.

When the VLIW *i* is to be executed, the *i*th row of ifnull array from FIR is read first, then the value of ifnull [*i*] [*j*] is judged respectively ($j = 1,2,\ldots\ldots, w$), as shown in Figure 8(b). If ifnull[*i*][*j*]=0, it represents that the field *j* of VLIW *i* is NOP, and the read address of this bank *j* is 0, (The value of 0 address register is 0, and it is the same as NOP); If ifnull[*i*][*j*]=1, it means that the field *j* is a valid operation. The address of this field in bank *j* is the result of offset *j* in FPCR adding to base address *j* in BAR, as shown in Figure 8(b). we set ifnull [] [] to 512 rows, which can satisfy most of kernels sizes. If an excessively large kernel exceeded this limit, further partitioning can be performed.

The horizontal position of every subfield is fixed and there isn't any subfield moved in horizontal direction in vertical compression. Furthermore, the vertical compression just deletes NOP without changing the width of valid operation, so the subfield width to be read is also invariant. This implies that all subfields in a VLIW are independent from each other, and thus they could be read in parallel. If the ifnull array read

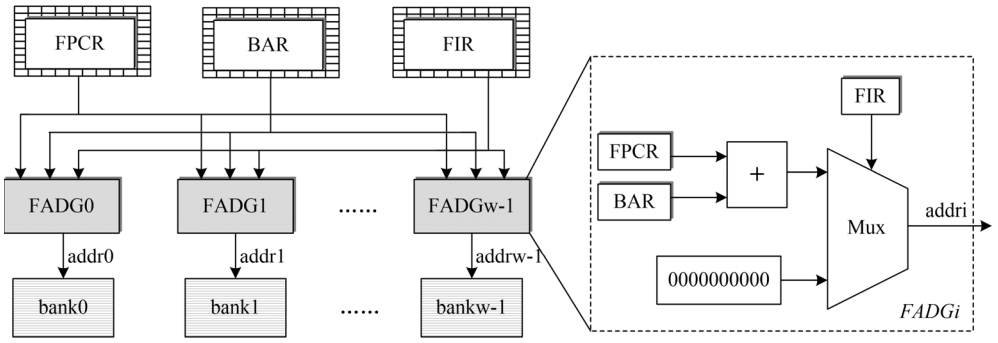



**FIGURE 7.** Architecture of subfield address generating unit.

operation and field read operation can be executed in pipeline, one VLIW can be fetched in a single cycle, which can match the VLIW issue speed. Consequently, vertical compressed instruction will not cause stop-wait problem.

## V. EXPERIMENTS AND RESULTS ANALYSES

### A. COMPRESSION RATIO

In order to analyze the performance of vertical compression scheme, six typical stream applications were mapped to CISP (Its structure can be seen in Figure 1), which is a stream processor prototype with five parallel function units and the vertical compression was applied to the VLIW. The CR was shown in Table 3.

**TABLE 3.** CR of vertical compression.

| Application name | CR(off-chip) | CR(on-chip) |
|---|---|---|
| Reed-Solomon | 45.92% | 26.04% |
| H.264 | 54.36% | 32.70% |
| Ygx2 | 48.64% | 28.82% |
| Susan | 41.28% | 29.61% |
| LUCAS | 47.73% | 30.09% |
| MPEG2 | 49.96% | 31.83% |

After vertical compression, the off-chip CR, which represents compressed instruction in external storage before they are loaded into memory, is about 50%, and in the best situation it can reach 41.28%. This is because vertical compression can delete all NOPs in VLIW, although NOPs were filled for storing, and ifnull and profile were added for decompression, the compressed VLIW still achieved more than 50% code size reduction on average.

When compressed VLIW is loaded to on-chip instruction memory, we can delete the filled NOPs, as the instruction memory is banked and each bank has its own address. The on-chip VLIW size can achieve a 70% reduction.

The vertical compression method is compared with other compression methods proposed in related works. We can see from Table 4 that when different compression methods are employed in the same stream applications, the off-chip and on-chip CR of vertical compression reach 47.98% and 29.85% on average respectively, which are both better than the compression method proposed by Guan *et al.* [34] This is due to that vertical compression deleted a large number of NOPs and generated only a little extra identity information.

**TABLE 4.** CR comparison of different compression methods.

| | Vertical Compression | [34] | [35] | [36] | [37] |
|---|---|---|---|---|---|
| Off-chip CR | 47.98% | 61.44% | — | — | — |
| On-chip CR | 29.85% | 34.58% | 52.8%/ 57.20%* | 77%/ 65.78%* | 55%-65% |

The CR of vertical compression is also better than the methods in [35] (52.8%) and [36] (77%).As the number and distribution situation of different applications are different, in order to justify comparison of the results, we applied compression methods of [35] and [36] to the six stream applications, and get the average CR statistic with "∗". The CR of vertical compression is far better than the methods in [35] and [36] with the same applications. We also compared vertical compression with [37] which evaluated nine embedded applications on three target architectures with bitmask-based code compression and gained compression ratio between 55% and 65%. The vertical compression can gain a significant CR as it divided the VLIW into several fields according to the VLIW logic characteristics.

### B. RESOURCE CONSUMPTION ANALYSIS

In order to analyze resource consumption of vertical compression, the address generating unit and instruction memory were synthesized with DC (Design Compiler) based on 65 nm CMOS process. Moreover, FPCR and BAR were implemented with register file, and instruction memory and FIR were implemented with RAM. The on-chip instruction memory capacity could be reduced to 50% because vertical compression reduced code size sharply. The synthesized result is shown in Table 5. Although address generating unit increased hardware consumption, it is still a low cost compared to the reduced instruction memory, and the instruction memory area and the whole system area are reduced by 44.26% and 9.52% respectively.

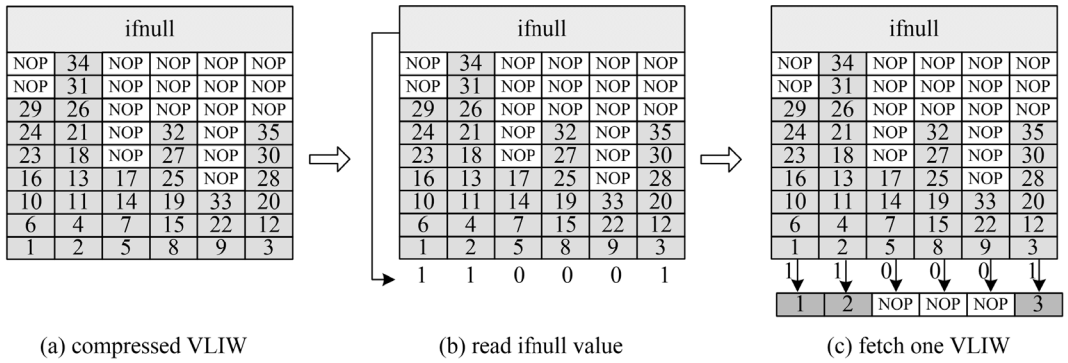



**FIGURE 8.** Fetching instruction procedure of vertical compression.

**TABLE 5.** Evaluation of resource consumption.

| | Pre-compression | Post-compression | | Area saving |
|---|---|---|---|---|
| Whole system | 833901 | 754476 | | 9.52% |
| Instruction memory | 179456 | Total | 100032 | 44.26% |
| | | ADG | 10304 | |
| | | Instruction memory | 89728 | |

## VI. CONCLUSION

As the VLIW width is large and the amount of NOP is huge, which cause the code size on stream processors expands seriously. In order to compress code size, Huffman coding is applied to compress VLIW in horizontal direction. The VLIW is divided into several subfields by its logical structure and every subfield is compressed by Huffman coding, then the code size can be reduced over 78% on average. Although the horizontal compression can gain excellent compression ratio, the compressed VLIW must be decoded in sequence, which leads to long decoding delay and affects the system performance dramatically. For this issue, vertical compression scheme is proposed. After compressed by vertical compression, the code size of VLIW is not only reduced by around 50%, the compressed VLIW can also be decompressed in a single cycle by all subfields of a VLIW decoded in parallel. As the amount and distribution of NOPs in different subfields are not same, so the future work is to modify compression method and instruction memory design to improve CR and reduce area of the instruction memory.

## REFERENCES


[1] S. Rixner, *Stream Processor Architecture*. Boston, MA, USA: Kluwer, 2002.

[2] C. Zhang, M. Wen, and N. Wu, *Research and Design of Stream Processor*. Beijing, China: Publishing House of Electronics Industry, 2009.

[3] M. Xu, H. An, and X. Tang, "A dataflow-like driven reconfigurable manycore stream processor," *J. Chin. Comput. Syst.*, vol. 34, no. 6, pp. 1359–1364, 2013.

[4] L. Affetti, A. Margara, and G. Cugola, "FlowDB: Integrating stream processing and consistent state management," in *Proc. 11th ACM Int. Conf. Distrib. Event-Based Syst. (DEBS)*, 2017, pp. 134–145.

[5] M. Michalska, S. Casale-Brunet, E. Bezati, M. Mattavelli, and J. Janneck, "Trace-based manycore partitioning of stream-processing applications," in *Proc. 50th Asilomar Conf. Signals, Syst. Comput.*, Nov. 2016, pp. 422–426.

[6] Y.-J. Kim, H.-E. Kim, S.-H. Kim, J.-S. Park, S. Paek, and L.-S. Kim, "Homogeneous stream processors with embedded special function units for high-utilization programmable shaders," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 20, no. 9, pp. 1691–1704, Sep. 2012.

[7] Y. Xiao, L. Liu, and S. Wei, "Design and implementation of reconfigurable stream processor in multimedia applications," in *Proc. Int. Conf. Commun., Circuits Syst.*, May 2008, pp. 1382–1386.

[8] J. Yan, D. Wu, and R. Wang, "Socially aware trust framework for multimedia delivery in D2D cooperative communication," *IEEE Trans. Multimedia*, vol. 21, no. 3, pp. 625–635, Mar. 2019.

[9] U. Scaiella, G. Berardi, G. Mega, and R. Santoro, "Sedano: A news stream processor for business," in *Proc. 39th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr. (SIGIR)*, 2016, pp. 525–526.

[10] J. Xiong, R. Ma, L. Chen, Y. Tian, Q. Li, X. Liu, and Z. Yao, "A personalized privacy protection framework for mobile crowdsensing in IIoT," *IEEE Trans. Ind. Informat.*, vol. 16, no. 6, pp. 4231–4241, Jun. 2020.

[11] D. Wu, B. Liu, Q. Yang, and R. Wang, "Social-aware cooperative caching mechanism in mobile social networks," *J. Netw. Comput. Appl.*, vol. 149, Jan. 2020, Art. no. 102457.

[12] G. Li, J. Xu, Y. Zhu, Z. Dai, S. Wang, and X. Feng, "Design of block cipher processor based on stream processor architecture," *J. Comput. Res. Develop.*, vol. 12, pp. 2833–2842, Dec. 2017.

[13] J. Xiong, M. Zhao, M. Bhuiyan, L. Chen, and Y. Tian, "An AI-enabled three-party game framework for guaranteed data privacy in mobile edge crowdsensing of IoT," *IEEE Trans. Ind. Informat.*, early access, Dec. 2, 2019, doi: 10.1109/TII.2019.2957130.

[14] V. K. Goyal, A. K. Fletcher, and S. Rangan, "Compressive sampling and lossy compression," *IEEE Signal Process. Mag.*, vol. 25, no. 2, pp. 48–56, Mar. 2008.

[15] X. Liang, Z. Li, Y. Yang, Z. Zhang, and Y. Zhang, "Detection of double compression for HEVC videos with fake bitrate," *IEEE Access*, vol. 6, pp. 53243–53253, 2018.

[16] H. Wu, X. Sun, J. Yang, W. Zeng, and F. Wu, "Lossless compression of JPEG coded photo collections," *IEEE Trans. Image Process.*, vol. 25, no. 6, pp. 2684–2696, Jun. 2016.

[17] M. A. Kabir and M. R. H. Mondal, "Edge-based transformation and entropy coding for lossless image compression," in *Proc. Int. Conf. Electr., Comput. Commun. Eng. (ECCE)*, Feb. 2017, pp. 717–722.

[18] W. R. Azevedo Dias and E. D. Moreno, "Code compression using Huffman and dictionary-based pattern blocks," *IEEE Latin Amer. Trans.*, vol. 13, no. 7, pp. 2314–2321, Jul. 2015.

[19] S. Wang, "Multimedia data compression storage of sensor network based on improved Huffman coding algorithm in cloud," *Multimedia Tools Appl.*, vol. 17, pp. 1–14, May 2019.

[20] W. Liu, E. Zhu, and J. Wang, "Optimization algorithm and VLSI design of the arithmetic coder in JPEG2000," *ACTA Electronica Sinica*, vol. 39, no. 11, pp. 2486–2491, 2011.

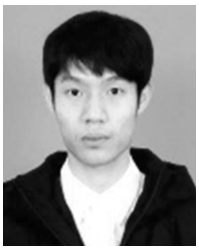
[21] A. Wolfe and A. Chanin, "Executing compressed programs on an embedded RISC architecture," in *Proc. 25th Annu. Int. Symp. Microarchit. (MICRO)*, 1992, pp. 81–91.

[22] Y. Yang, "Research on code compression for embedded processors," Ph.D. dissertation, Dept. Elect. Eng., Zhejiang Univ., Hangzhou, China, 2007.

[23] W. R. A. Dias, E. David Moreno, and I. N. Palmeira, "A new code compression algorithm and its decompressor in FPGA-based hardware," in *Proc. 26th Symp. Integr. Circuits Syst. Design (SBCCI)*, Sep. 2013, pp. 1–6.
[24] T. Bonny and J. Henkel, "Huffman-based code compression techniques for embedded processors," *ACM Trans. Design Autom. Electron. Syst.*, vol. 15, no. 4, pp. 1–37, Sep. 2010.
[25] M. A. Rahman, S. M. S. Islam, J. Shin, and M. R. Islam, "Histogram alternation based digital image compression using Base-2 coding," in *Proc. Digit. Image Comput., Techn. Appl. (DICTA)*, Dec. 2018, pp. 1–8.
[26] M. Rahman, J. Shin, and A. Saha, "A novel lossless coding technique for image compression," in *Proc. Joint 7th Int. Conf. Informat., Electron. Vis. (ICIEV) 2nd Int. Conf. Imag., Vis. Pattern Recognit. (ic4PR)*, 2018, pp. 82–86.
[27] J. Wei, W. Guo, J. Sun, and Y. Yao, "Program compression based on arithmetic coding on transport triggered architecture," in *Proc. Int. Conf. Embedded Softw. Syst. Symp.*, Jul. 2008, pp. 126–131.
[28] J. Voges, T. Paridaens, F. Müntefering, L. S. Mainzer, B. Bliss, M. Yang, I. Ochoa, J. Fostier, J. Ostermann, and M. Hernaez, "GABAC: An arithmetic coding solution for genomic data," *Bioinformatics*, vol. 36, no. 7, pp. 2275–2277, Apr. 2020.
[29] A. R. Buzdar, L. Sun, A. Latif, and A. Buzdar, "Instruction decompressor design for a VLIW processor," *J. Microelectron., Electron. Compon. Mater.*, vol. 45, no. 4, pp. 225–236, Jan. 2015.
[30] H. Wang, T. Wang, L. Liu, H. Sun, and N. Zheng, "Efficient compression-based line buffer design for Image/Video processing circuits," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 10, pp. 2423–2433, Oct. 2019.
[31] D. Lemire, N. Kurz, and C. Rupp, "Stream VByte: Faster byte-oriented integer compression," *Inf. Process. Lett.*, vol. 130, pp. 1–6, Feb. 2018.
[32] L. Acasandrei and M. Neag, "A fast parallel Huffman decoder for FPGA implementation," *ACTA Technica Napocensis*, vol. 49, no. 1, pp. 8–15, 2008.
[33] Y. Zhou, H. Ge, and J. Lin, "Entropy coding techniques and protocol to support parallel processing with low latency," *Microcomput. Appl.*, vol. 32, no. 11, pp. 84–86, 2013.
[34] M. Guan, Y. He, and Q. Yang, "Optimized design research of instruction memory for stream architecture," *ACTA Electronica Sinica*, vol. 40, no. 7, pp. 1379–1386, 2012.
[35] Z. Ji, X. Chen, and J. Xu, "Research and implementation of VLIW compressing technology based on instruction prefix," *Appl. Electron. Technique*, vol. 39, no. 4, pp. 22–25, 2013.
[36] T. Jin, M. Ahn, D. Yoo, D. Suh, Y. Choi, D.-H. Kim, and S. Lee, "Nop compression scheme for high speed DSPs based on VLIW architecture," in *Proc. IEEE Int. Conf. Consum. Electron. (ICCE)*, Jan. 2014, pp. 304–305.
[37] S.-W. Seong and P. Mishra, "A bitmask-based code compression technique for embedded systems," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design*, Nov. 2006, pp. 251–254.

**GONGLI LI** born in 1981. She received the Ph.D. degree from Information Engineering University, in 2018. Her main research interests include cryptographic arithmetic, computer architecture, and high-performance cryptographic processors.

**YINGYING HOU** received the bachelor's degree in computer and information engineering from Henan Normal University, in 2016, where she is currently pursuing the master's degree in computer and information engineering. Her interests are in the fields of cryptography and information security.



**JUNZHE ZHU** received the bachelor's degree in computer and information engineering from Henan Normal University, in 2017, where he is currently pursuing the master's degree in computer and information engineering. His research interests include cryptography and information security.